Apr 16, 2026

ASPLOSer Modeling Language

- Simple structure, rich attributes and mechanisms
  - Threat Model
  - Elements
  - Mechanisms
  - Granularity

Threat Model

1. Threat Model
    a. Internal threats
    b. External threats

Threat Model: Internal threats

- Confidentiality:
  - Participants of the IS life cycle may exploit object information that they are presumed not to leverage.
- Integrity:
  - Participants of the ISLC may deliberately undermine the correctness of the ISLC process.
  - The ISLC outcomes may be indeliberately damaged due to the lack of expertise or the low quality of initial assets.
- Availability:
  - Participants or agents may not continuously carry out processing due to internal problems of the participant teams, low-quality transmission, and low-quality assets.
- Subjects can be:
  - Curious but honest (low credibility and high correctness)
  - Incurious but dishonest (high credibility and low correctness)
  - Curious and dishonest (low credibility and low correctness)

Threat Model: External threats

- Attackers will exploit any risk in the ISLC, regardless of whether the risk relates to confidentiality, integrity, or availability.

Elements

2. Elements
    a. Structural assumptions
    b. Attribute
    c. Trust level
    d. Subject
    e. Action
    f. Object
    g. Metrics
    h. Sub-attribute and sub-metrics

Elements: Structural assumptions

- In each development sub-model, alpha tests carried out by developers have been involved in development actions.
  - There is no feedback and refinement without explicit interactive feedbacks inside each development sub-model.
  - That is to say, correctness and continuity problems will not be reported and solved until the development process has ended, or until the next ISLC loop start.

Elements: Attribute

- Confidentiality
  - Credibility: the willingness to preserve secrecy of the objects that the subject is holding
    - Subjects
  - Concealment: the ability to prevent information leakage of the object that the action is processing
    - Actions
  - Ciphering: the ability of cipher or decipher objects
    - Subjects
  - Ciphered: whether an *object* is ciphered or not. *granularity*
    - Objects
    - When an object is ciphered, low confidentiality or credibility will not result in leakage of the object information (external attackers or ? is the data confidential to the OS that hold the data)
- Integrity
  - Correctness: the ability to prevent objects that are held, processed, or produced from tampering or faults
- Availability
  - Continuity: the ability to continuous hold, process, or produce objects without being interrupted
- Quantification
  - Criticality: the numerical value (calculated from CVSS or other security cases) that represents the security significance of the link

Elements: Trust level

- Each attribute has three levels of trust: high, middle, and low.
- High: the corresponding ability of the element is totally trusted, which is expected not to result in any future security incidents.
- Middle: the corresponding ability of the element is not totally trusted, which has not resulted in any security incidents yet.
- Low: the corresponding ability of the element has led to security incidents.

Elements: Subject

- Definition: nodes that hold and send objects actions
- Type: DataWorkers, ModelDevs, AppDevs, DepDevs, Maintainers, Users, ExternalEnv, PreprocessingModule, InferenceModule, PostprocessingModule
  - Only ExternalEnv cannot send feedback.
  - The attribtue values of IS modules, which are subjects, are inferenced from corresponding *raw materials*
    - Pre-processing module and post-processing module: application and corresponding dependency
    - Inference module: model, application, and corresponding dependency
- Attributes:
  - Credibility
  - Ciphering
  - Correctness
  - Continuity
  - Criticality
  - Attribute values are designated by model users, except for those of IS modules.
- Notes:
  - Subjects can be AI agents.
  - Different subjects can be identical participants in reality, with same attribute values or not.

Elements: Action

- Definition: nodes that process objects and send the outcomes to subjects
- Type: ModelAction, AppAction, DepAction, PrepareAction, OpAction, FeedbackAction
  - Actions of the same type shall proceed according to their numerical/topological order
- Attributes:
  - Concealment
  - Correctness
  - Continuity
  - Criticality
  - Attribute values are designated by model users, except for those of actions sent from IS modules.
- Notes:
  - Initialization actions have no sender subjects and their attribute values cannot be designated, which are fixed in the highest trusty.
  - The existence of feedback actions can be customized.
  - The position of initialization actions can be customized, with former actions being eliminated in the instance.
  - The confidentiality and concealment of subjects and actions may be considered respectively when taking confidential computing into account.

Elements: Object

- Definition: edges that represent assets provided to the IS life cycle and assets processed in the cycle.
- Type: Priori, Input, Output
  - Priori: objects that come from initialization actions, whose attribute values must be designated in advance
  - Input: objects sent to subjects
  - Output: objects sent out by subjects
- Attributes:
  - Ciphered
  - Correctness
  - Continuity
  - Criticality
  - Attribute values are inferred from the prior object and subject or action, except for those of Priori objects.

Elements: Metrics

- Objectives:
  - Confidentiality: the ability to preserve secrecy of digital assets
  - Integrity: the ability to produce correct inference outcomes
  - Availability: the ability to continuously operate without being interrupted
- Risk: originated from the dissatisfaction of each objective
  - Calculated per node
  - Designated: the risk is inherent due to the designated low-level trust of elements
  - Inferred: the risk originates from attribute inference rules
  - Propagated: the risk originates from risk propagation rules
- Qualitative: the existence of risks
  - Calculated with inference rules in the model graph
- Quantitative: the criticality of risks
  - Frequency and influence of the risk
  - Calculated with CVSS and other datasets that provide empirical basics of the numerical quantification of risks

Elements: Sub-attribute set and sub-metrics

- Sub-attribute set: parallel attribute set that is identical to generic attribute set (Credibility, Correctness, …), which is leveraged to calculate metrics in a specific area
  - NetworkConnectivity:
    - Confidentiality: the ability to prevent object information leakage that resulted in with the network
    - Integrity: the ability to prevent object from being tampered with the network
    - Availability: the ability to hold, process, or receive objects without being interrupted by network failures
- Sub-metrics: metrics that are the union of specific generic metrics
  - Trusty: Confidentiality & Integrity

Mechanisms

3. Mechanisms
    a. Attribute assignment
    b. Risk propagation
    c. Dynamic analysis
    d. Customization

Mechanisms: Attribute assignment

- Designated attributes:
  - most subjects (excluding IS modules)
  - initial objects
  - most actions (excluding actions generated by IS modules)
- Inferred-only attributes:
  - IS module subjects
  - most objects (excluding initial objects)
  - actions generated by IS modules
- Attributes that can propagate:
  - Correctness
  - Continuity
  - Ciphered
- Notes:
  - Confidentiality (for subjects) and concealment (for objects) cannot propagate.

Mechanisms: Risk propagation

- Applicable attributes: correctness and continuity
- When an object has lower attribute levels, the designated corresponding attribute levels of the subsequent subject or action will be downgraded to the levels of the object attributes.
- Such downgration will cause risk propagation

Mechanisms: Dynamic analysis

- Step-by-step simulation
  - The order of executing model, application, and dependency submodel instances can be designated by users.
- Feedback
  - Reporting: subjects can report correctness and continuity incidents they have found to corresponding subjects.
  - Intra-loop fix: after reporting, the risk origin will be immediately fixed before its execution.
  - Inter-loop fix: after reporting, the risk origin will be fixed only when simulation of the whole loop has ended.
  - Best-effort fix: risk-related attributes will be promoted to the level of the fixer’s attribute.
  - Faultless fix: risk-related attributes will be promoted to the highest level.
- Two-loop simulation
- Notes:
  - No feedback/filter inside each sub-model, which can be redundant and improper

Mechanisms: Customization

- Feedback
  - The existence of feedback actions can be customized.
- Initialization
  - The position of initialization actions can be customized, with former actions being eliminated in the instance.
- Multiple models, application, and dependencies
  - Users can define several dependencies, with each one connected to corresponding IS modules.

Granularity

ASPLOSer := {ISLC_graph, MechanismSet}

ISLC_graph := {Nodes, Edges}

0. Black box
    a. Nodes := {Sender, Computation, Receiver}
    b. Edges := {InitialAssets, Input, Output}
1. Sub-systems
    a. Nodes := {ModelSubprocess, AppSubprocess, DepSubprocess, PrepareSubprocess, OperationSubprocess, FeedbackSubprocess}
    b. Edges := Actionset
2. The generic model
3. API?

Prior models: only targeting one granularity (sub-system level or API/technique level)

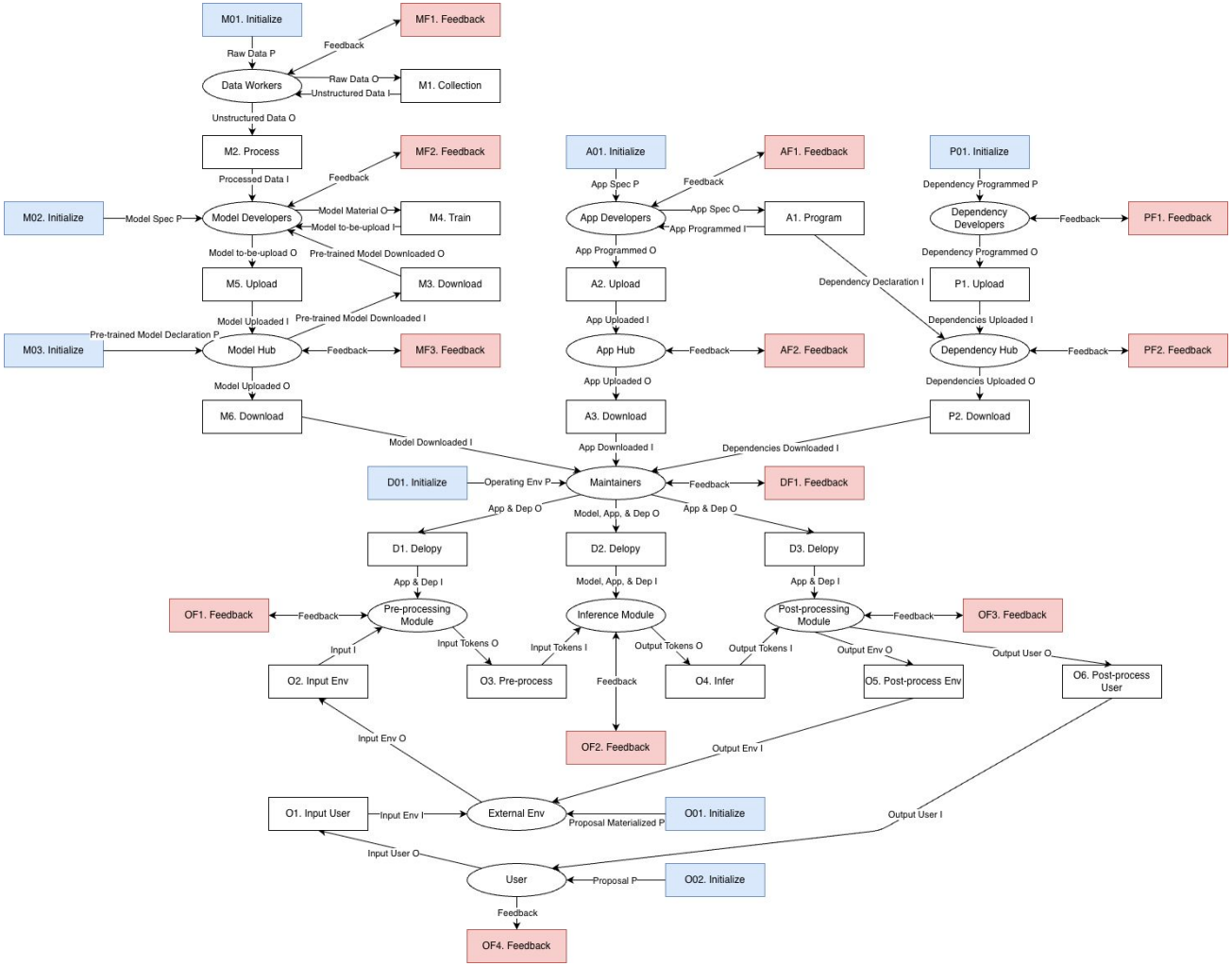
Challenges

- Formal documentation
- Granularity
- Emphasis on model *(IS)* properties
  - Model attack surface
  - Model capability
  - Model types
  - With adversarial examples = System safety = Model*adversarial
  - OR without adversarial: system safety = model
    - Which can be reasoned by the graph model
  - THAT is to say, we need a fine granularity model
    - For techniques, attacks, and risks

Use Cases

1. Dynamic analysis of the IS
    a. Why need?
        i. Static analysis may produce false positives and does not reflect the actual deployment of the IS.
    b. Goal
        i. To help find new bugs/risks/IS-focused vulnerabilities (not software or memory bugs!!). NEED to think about: why prior arts fail to find them??
    c. Key use cases and benefits:
        i. To build “IS” sanitizer. Like a memory sanitizer, which identifies potential risks and information leakage.
            1. Metrics: # of risks
        ii. To build “IS fuzzer”. For many unassigned nodes or those developers are not so sure, system automatically discovers potential 0-days.
            1. Metrics: # of 0-days
2. Thorough auditing of the IS
    a. Why need?
        i. Allow targeted/directional testing/benchmarking of the IS
        ii. Need to think: why prior arts fail to do so?
    b. Key use cases:
        i. Filter out sensitive/most tangible attack surfaces (e.g., adversarial samples, prompt injection) and conduct corresponding evaluations. This skips many unnecessary benchmarks and save time.
            1. Metrics: execution latency
        ii. Allow developers to reason about the safety of the IS
            1. Metrics: user study (???)

Apr 23, 2026


M01. Initialize
MF1. Feedback
Raw Data P
Feedback
Data Workers
Raw Data O
Unstructured Data I
M1. Collection
Unstructured Data O
M2. Process
MF2. Feedback
Processed Data I
Feedback
M02. Initialize
Model Spec P
Model Developers
Model Material O
Model to-be-upload I
M4. Train
Model to-be-upload O
Pre-trained Model Downloaded O
M5. Upload
M3. Download
Model Uploaded I
Pre-trained Model Downloaded I
Pre-trained Model Declaration P
M03. Initialize
Model Hub
Feedback
MF3. Feedback
Model Uploaded O
M6. Download
A01. Initialize
AF1. Feedback
App Spec P
Feedback
App Developers
App Spec O
A1. Program
App Programmed I
App Programmed O
A2. Upload
App Uploaded I
App Hub
Feedback
AF2. Feedback
App Uploaded O
A3. Download
P01. Initialize
Dependency Programmed P
Dependency Developers
Feedback
PF1. Feedback
Dependency Programmed O
Dependency Declaration I
P1. Upload
Dependencies Uploaded I
Dependency Hub
Feedback
PF2. Feedback
Dependencies Uploaded O
P2. Download
Model Downloaded I
App Downloaded I
Dependencies Downloaded I
D01. Initialize
Operating Env P
Maintainers
Feedback
DF1. Feedback
App & Dep O
Model, App, & Dep O
App & Dep O
D1. Delopy
D2. Delopy
D3. Delopy
App & Dep I
Model, App, & Dep I
App & Dep I
OF1. Feedback
Feedback
Pre-processing Module
Inference Module
Post-processing Module
Feedback
OF3. Feedback
Input I
Input Tokens O
Input Tokens I
Output Tokens O
Output Tokens I
Output Env O
Output User O
O2. Input Env
O3. Pre-process
Feedback
O4. Infer
O5. Post-process Env
O6. Post-process User
Input Env O
OF2. Feedback
Output Env I
O1. Input User
Input Env I
External Env
O01. Initialize
Proposal Materialized P
Output User I
Input User O
User
Proposal P
O02. Initialize
Feedback
OF4. Feedback

Threat Model and Metamodel

1. Threat models in typical CS papers
    a. Identify concrete threat entities in the specific system presented in the paper
    b. Do not discuss the *ontological* definitions of adversaries, threats, attacks, and other concepts
    c. Have correctness/trust assumptions of the system
2. Metamodels
    a. Identify different types of threats in generic software systems
    b. Give ontological defintions of adversaries, threats, attacks, and other concepts
    c. Do not have assumptions of the structure or correctness of software systems

Definitions

1. Adversaries
    a. Internal adversaries: subjects with low-level security attributes
    b. External adversaries: attackers outside the SDLC
2. Vulnerability: elements with low-level attributes
3. Threats (risks): elements with low-trust attributes *that are not confirmed as features of the SDLC*
    a. E.g: open-source software has low-level concealment that is confirmed as a feature (publicity)
4. Attacks: attempts of malicious activities against the SDLC
    a. Attacks must leverage low-level security attributes of the SDLC to achieve success
    b. Successful attacks will compromise the CIA triad of the SDLC
5. Attack surface
    a. Internal adversaries: network, hardware, software, and humans of the next subject/object/action.
    b. External adversaries: network, hardware, software, and humans of the whole SDLC

Granularity

1. Structure (nodes and edges)
    a. Current: Model, Application, Dependency, Preparation, Operation, and Feedback systems with multiple internal nodes and edges
    b. Coarser: systems without internal
    c. Finer: sub-modules, APIs, …
2. Types (additional systems)
    a. Current: intelligent systems with model-development system
    b. Coarser: generic software systems
    c. Finer: LLM chatbot, on-device image identification, … (more dependencies, …)
3. Component (attributes)
    a. Current: six attributes corresponding to the CIA triad, one attribute for quantification
    b. Coarser: three attributes corresponding to the CIA triad
    c. Finer: techniques, network connection, cloud or on-device, …

Runtime

1. Signature of imported modules (dependencies)
2. Instrumentation of the dependency runtime

Benefits

1. A uniform way to present threats in software systems (**subset of IS**, the structure)
    a. SDLC/supplychain
    b. subjects/agents
    c. Formal language vs natural language (vague)
2. Custom granularity of the threat model
    a. Custom analysis granularity
3. Deducible and quantifiable safety and threats from the initial assumption of developers
4. Runtime that confirms or overthrows the assumption
    a. Missing spot
5. Decreased development efforts of security reinforcement